The Complete Frontend Dependency Chain

JSX • React • Babel • Vite • Node.js • npm • npx • nvm • Browser • V8 • {{ }}

This document ties the whole stack together: what runs where, why each piece exists, and how your code moves from JSX in your editor to pixels on the screen.

Big picture

Modern frontend development is a pipeline. You write developer-friendly code (React + JSX). Tools running on Node.js (Vite and Babel or equivalent transformers) turn it into plain JavaScript. The browser downloads that JavaScript and executes it using its JavaScript engine (V8 in Chrome/Edge). React then drives DOM updates, and the browser renders pixels.

What each piece does

Component	Where it runs	What it does
nvm	Your machine (shell)	Manages multiple Node.js versions and lets you switch per project.
Node.js	Your machine	JavaScript runtime outside the browser; executes build tools and scripts.
npm	Node tool	Installs packages into node_modules and runs scripts defined in package.json.
npx	Node tool	Runs a package binary without global installs (often used for scaffolding).
Vite	Node process	Dev server + build tool; serves modules in dev, bundles/optimizes in build.
Babel	Node process (via tooling)	Transforms JSX + modern JS into plain JS (and can downlevel syntax for older targets).

Continued

Component	Where it runs	What it does
JSX	Developer syntax	Syntax sugar that compiles to JavaScript calls. JSX never runs in the browser as-is.
React	Browser (runtime lib)	Component model + state + rendering logic; computes what UI changes are needed.
Browser	User device	Downloads assets; provides DOM, Web APIs, rendering, and networking.
V8 engine	Browser + Node	Parses, compiles, and executes JavaScript. Only understands JavaScript.
DOM + Renderer	Browser	Applies UI changes and paints pixels to the screen.

JSX: what it is (and what it is not)

JSX is not HTML. It’s JavaScript syntax sugar that makes UI code readable. Neither the browser nor V8 can execute JSX. JSX must be transformed into plain JavaScript first.

Example

```
// What you write (JSX)
const App = () => <h1>Hello</h1>;

// What tooling turns it into (conceptually)
const App = () => React.createElement("h1", null, "Hello");
```

By the time your code reaches the browser, JSX is already gone.

React: what it does at runtime

React runs in the browser as a library. It keeps track of component state and decides what should change in the UI when state or props change. Those decisions become DOM operations.

React does not start dev servers. React does not compile JSX. React does not bundle files.

Babel: the translator step

Babel is a compiler. It reads your source code and outputs JavaScript that a JavaScript engine can execute. In React projects, Babel is commonly used for JSX transformation. It can also transform newer JavaScript features into older equivalents depending on your target environments.

Important: Babel runs during development/build time inside Node.js, not inside the browser.

Vite: the orchestrator

Vite runs as a Node process. In development, it serves modules fast and transforms on demand (only what the browser requests). It supports fast refresh (HMR), so changes appear instantly. In production, it builds optimized static files for deployment.

Node.js, npm, npx, nvm: why they exist

Frontend tooling is mostly JavaScript programs. Those programs need a runtime on your machine: Node.js. npm downloads and manages them. npx runs package binaries conveniently. nvm keeps Node versions manageable across projects.

Typical flow

```
nvm use 20
npm install
npm run dev
```

Browser + V8: where your app actually runs

The browser downloads JavaScript and executes it in its JS engine (V8 in Chrome/Edge). That JavaScript uses DOM APIs and Web APIs to update the page. The browser’s rendering engine paints pixels.

Key boundary: build tools (Vite/Babel/npm) do not ship to the user as running processes. The user receives the output files.

Full dependency flow (text)

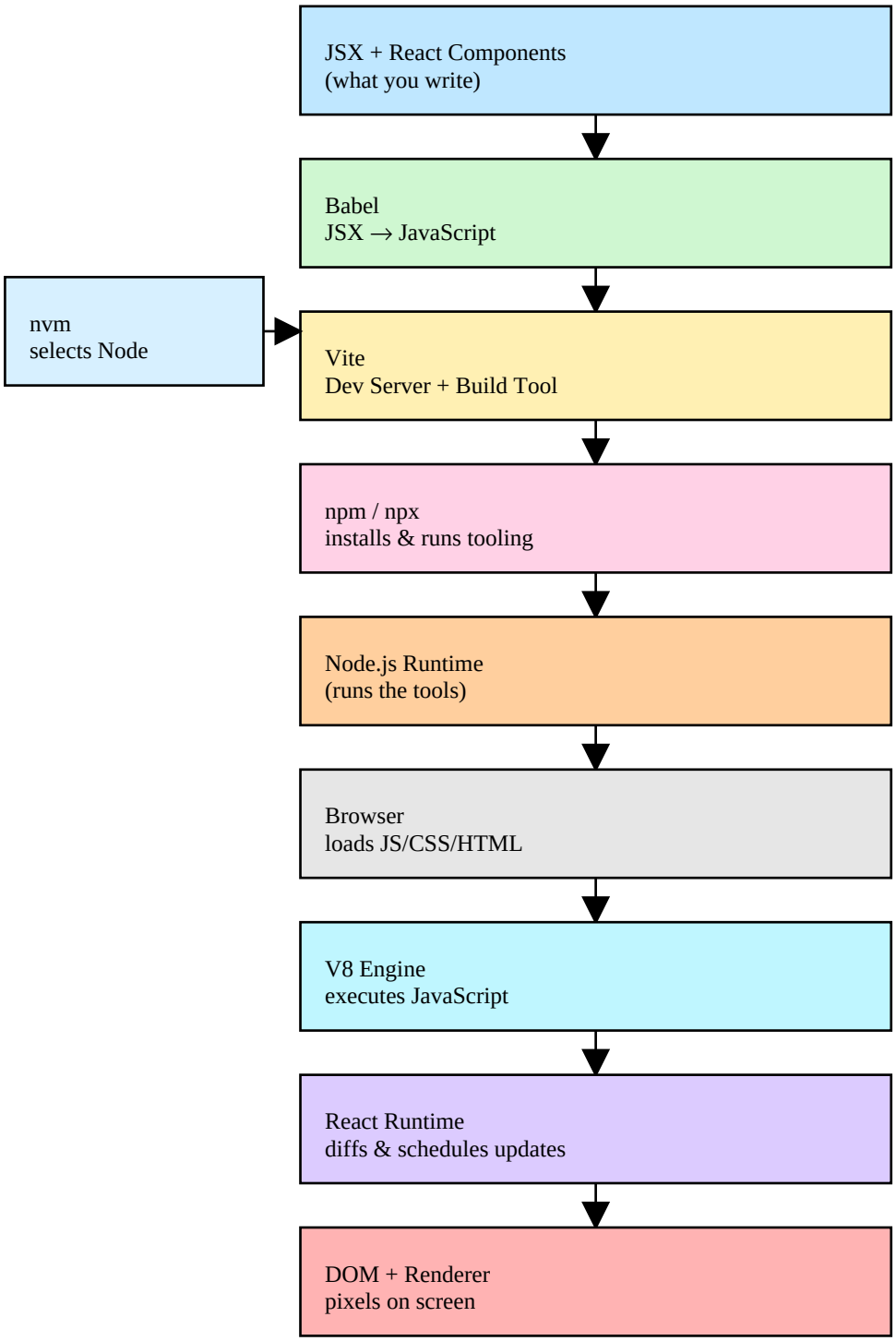
```
Developer writes:
  React components + JSX

Build-time / Dev-time (runs on your machine in Node.js):
  nvm -> selects Node version
  Node.js -> runs tooling
  npm/npx -> installs/runs scripts
  Vite -> dev server & build orchestration
  Babel (and/or other compilers) -> JSX + modern JS -> plain JS

Runtime (runs on the user’s device in the browser):
  Browser downloads JS/CSS/HTML
  V8 executes JavaScript
  React runs component logic
  DOM updates happen
  Rendering engine paints pixels
```

Colorful component diagram

Top boxes are build/dev-time (Node.js). Bottom boxes are runtime (Browser). Arrows show the direction of responsibility.



JSX double braces {{ }} (the real meaning)

In JSX, a single pair of braces { } means: switch from JSX into JavaScript. Inside braces, you can write any valid JavaScript expression.

So {{ }} is not special React syntax. It just means: a JavaScript object literal inside a JavaScript expression.

Example: inline styles

```
// JSX
<div style={{ color: "red", backgroundColor: "black" }}>
  Hello
</div>
```

How Babel translates it (conceptually)

```
React.createElement(
  "div",
  { style: { color: "red", backgroundColor: "black" } },
  "Hello"
);
```

After transformation, the double braces disappear because JSX is gone. Only the JavaScript object remains.

Where Vite fits

Vite doesn’t interpret {{ }}. Vite orchestrates the pipeline and serves transformed JavaScript to the browser.

```
Browser → Vite → Babel/esbuild → JavaScript → Browser
```

What happens when you run npm commands

npm run dev (development)

• Your shell uses nvm (if configured) to pick the Node version.

• npm reads package.json and runs the dev script (usually starts Vite).

• Vite starts a dev server and serves modules to the browser.

• When the browser requests a module, Vite transforms it (JSX removal, etc.).

• The browser executes the resulting JavaScript via V8, React updates the DOM, and the renderer paints pixels.

npm run build (production)

• npm runs Vite in build mode.

• Vite bundles and optimizes into static assets (HTML/CSS/JS), often in dist/.

• You deploy those files to a server/CDN.

• Users download them; the browser executes them via V8.

One-sentence mental model

nvm chooses Node. Node runs the tooling. npm supplies it. Vite orchestrates. Babel translates. The browser loads. V8 executes. React updates. The renderer paints.

React – 30 Day Incremental Learning Guide

This is your living React notebook built from the LDAP User Search project. Each day is preserved, appended, and explained with flow diagrams. Nothing is removed as new days are added.

30-Day Learning Plan Overview

Days 1–4: Core Mental Model

1. React bootstrapping and first render
2. JSX → Babel → createElement
3. Components, props, one-way data flow
4. State, controlled inputs, filtering, reconciliation

Days 5–10: Effects and Data Flow

1. useEffect and side effects
2. Data loading (static → async)
3. Loading and error states
4. Debouncing and cleanup

What You Have Learned Since Day 1

1. React mounts into a single root DOM node.
2. Components are plain JavaScript functions.
3. JSX is compiled at build time.
4. Props enforce one-way data flow.
5. Keys guide list reconciliation.
6. State changes trigger re-renders.
7. Controlled inputs bind DOM to state.
8. Filtering runs during render.
9. useEffect runs after DOM commit.

Day 1 – React Bootstrapping & Mental Model

React starts from index.html and mounts into a single root element. main.jsx creates the root and triggers the first render. Rendering means describing UI, not touching the DOM directly.

Flow Diagram

```
Browser loads index.html
    ↓
Vite serves main.jsx
    ↓
React.createRoot(...)
    ↓
<App /> renders (App() runs)
    ↓
Virtual DOM built
    ↓
Browser DOM updated
```

Day 2 – JSX, Lists, and Keys

JSX is JavaScript syntax, not HTML. Babel removes JSX at build time. Lists are rendered using map(), and keys give identity for reconciliation.

Flow Diagram

```
JSX written
    ↓
Babel transforms JSX
    ↓
React.createElement objects
    ↓
Array.map() builds list
    ↓
Keys identify elements
    ↓
Reconciliation updates DOM
```

Day 3 – Components and Props

You split responsibilities: App owns data and logic, UserRow renders UI. Props are read-only and flow downward.

Flow Diagram

```
App owns state and logic
    ↓
Props passed to UserRow
    ↓
UserRow renders UI
    ↓
No mutation, predictable output
```

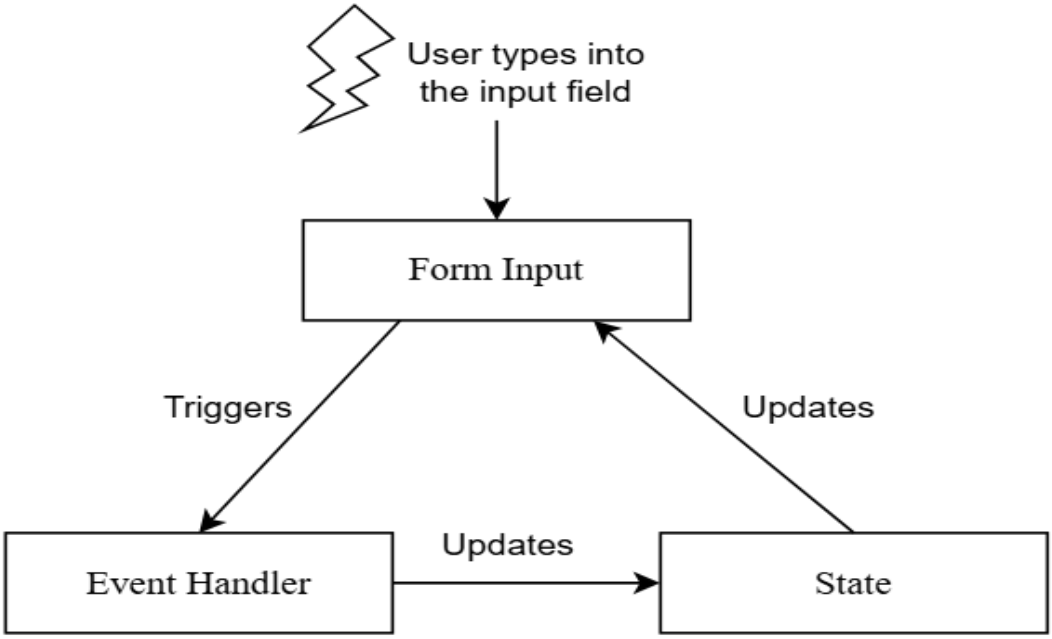
Day 4 – State, Controlled Inputs, and LDAP Query Flow

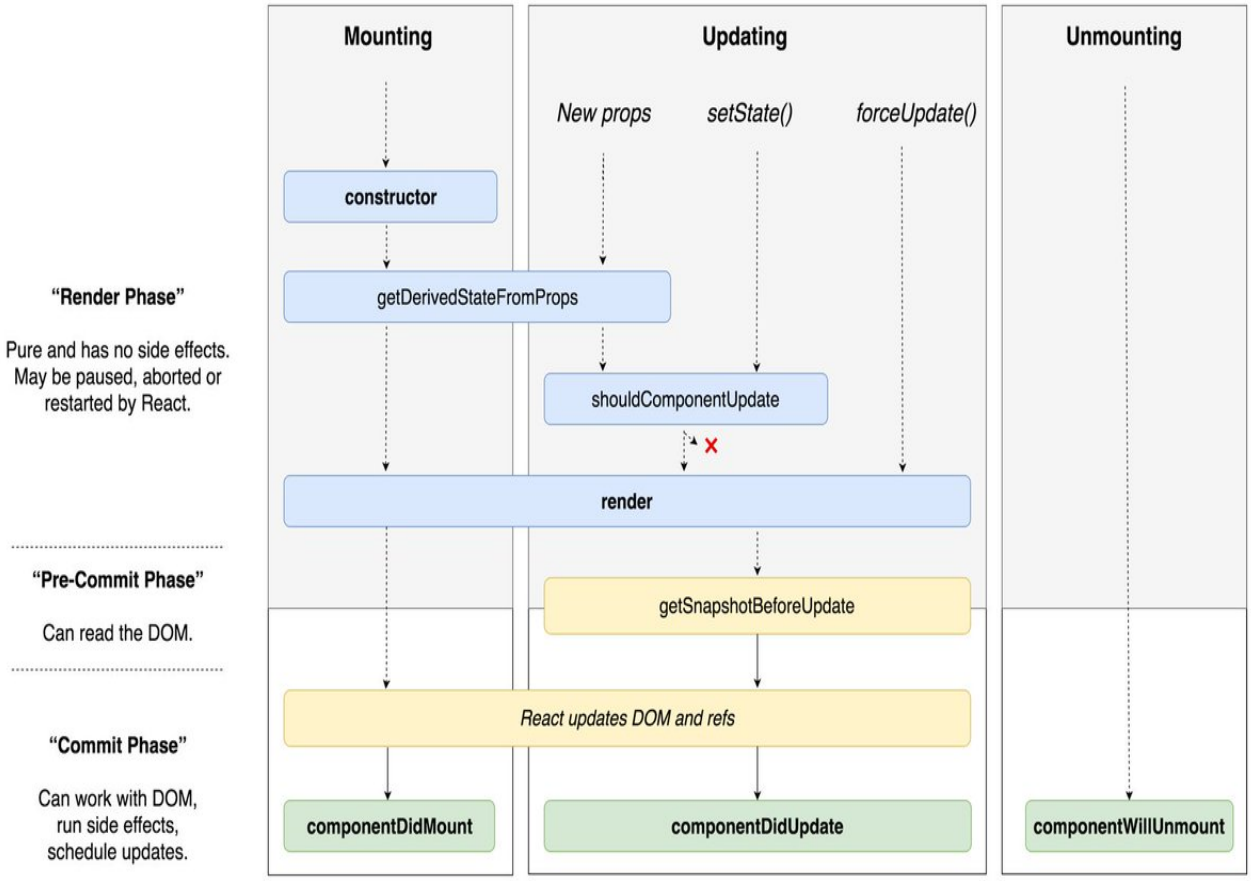
Typing in the search box updates state, which triggers re-rendering. Filtering runs during render and React reconciles efficiently.

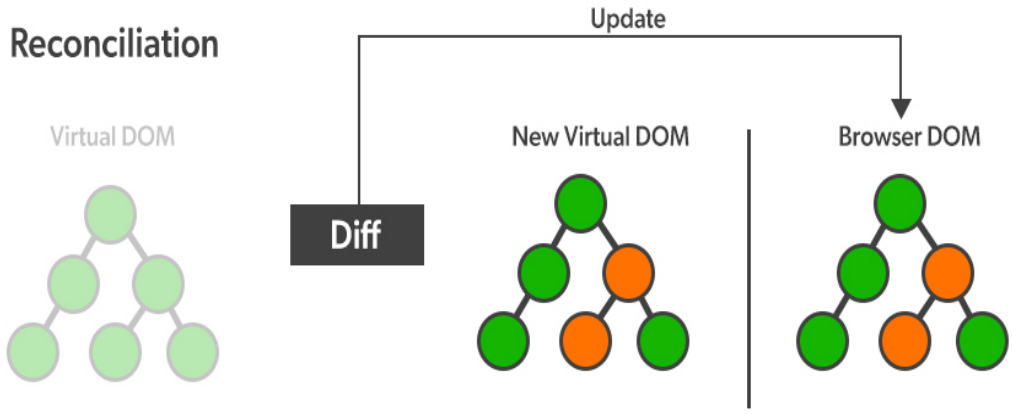
Execution Flow

```
User types character
    ↓
onChange handler fires
    ↓
setQuery(...) updates state
    ↓
App() re-runs
    ↓
filter() executes during render
    ↓
React reconciles DOM
```

Visual Flow




Mounting
Updating
Unmounting
New props
setState()
forceUpdate()
constructor
getDerivedStateFromProps
shouldComponentUpdate
render
getSnapshotBeforeUpdate
React updates DOM and refs
componentDidMount
componentDidUpdate
componentWillUnmount
"Render Phase"
Pure and has no side effects. May be paused, aborted or restarted by React.
"Pre-Commit Phase"
Can read the DOM.
"Commit Phase"
Can work with DOM, run side effects, schedule updates.


Reconciliation
Update
Virtual DOM
Diff
New Virtual DOM
Browser DOM

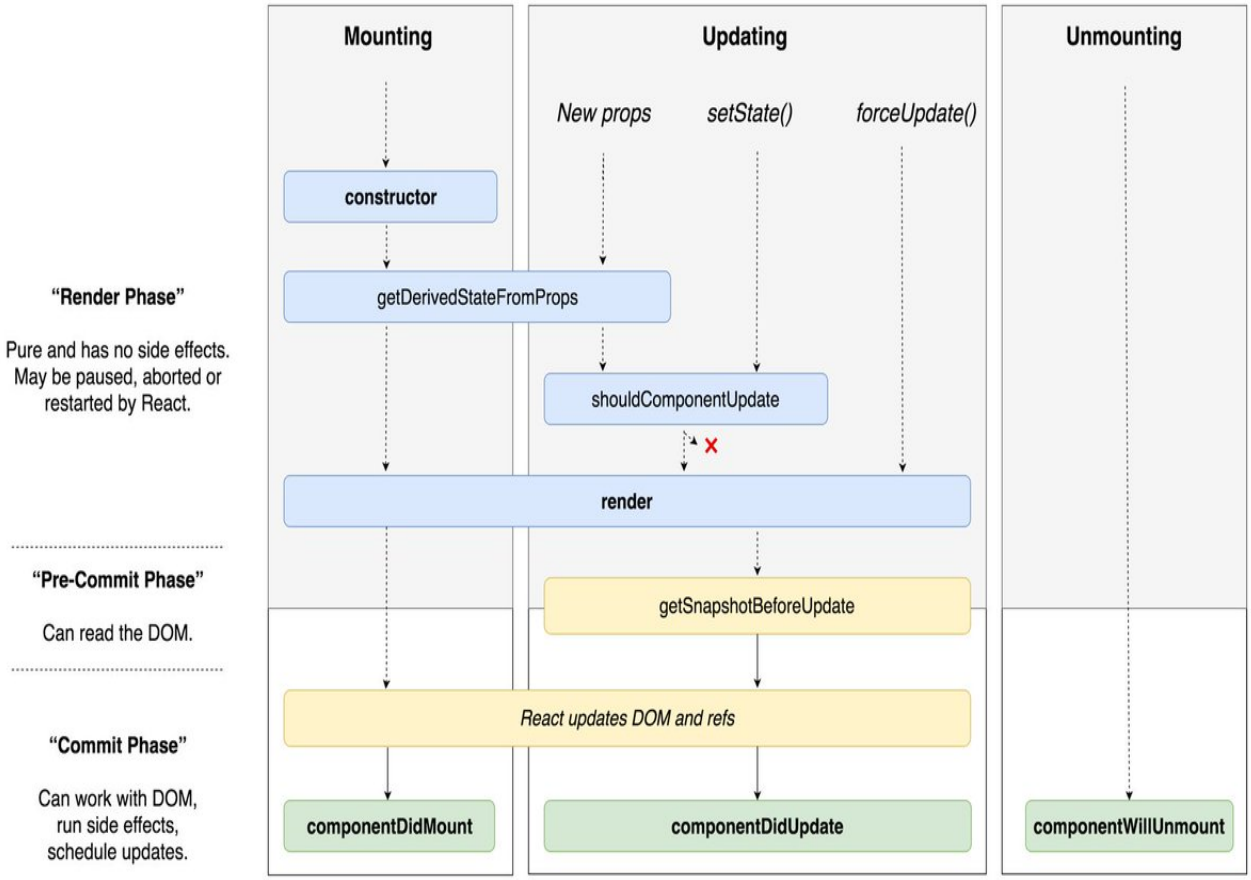
Day 5 – Side Effects and useEffect

Day 5 introduces time and the outside world. Rendering stays pure. Side effects must run after the DOM is committed. useEffect enforces this separation.

Flow Diagram

```
Render phase (pure)
    ↓
Commit phase (DOM painted)
    ↓
Effect phase (useEffect runs)
    ↓
setState inside effect
    ↓
Re-render scheduled
    ↓
DOM updated again
```

Visual Lifecycle



React – 30 Day Incremental Learning Guide

This is your living React notebook built incrementally using the LDAP User Search project. Each day is preserved and extended with explanations and flow diagrams.

Day 6 – Dependency Arrays, Effect Re-runs, and Render Timing

Day 6 explains when and why useEffect runs again. The dependency array does not schedule effects; it defines when React should re-run them by comparing previous and current values.

Effects never trigger rendering by themselves. Only state updates trigger renders. Derived values such as filteredUsers are always evaluated during render, never during effects.

Key Mental Rules

1. Components re-run top to bottom on every render.
2. useMemo and filtering logic run during render.
3. useEffect runs only after the DOM commit phase.
4. Dependency arrays are compared by reference.
5. Derived values must not be placed in dependency arrays.

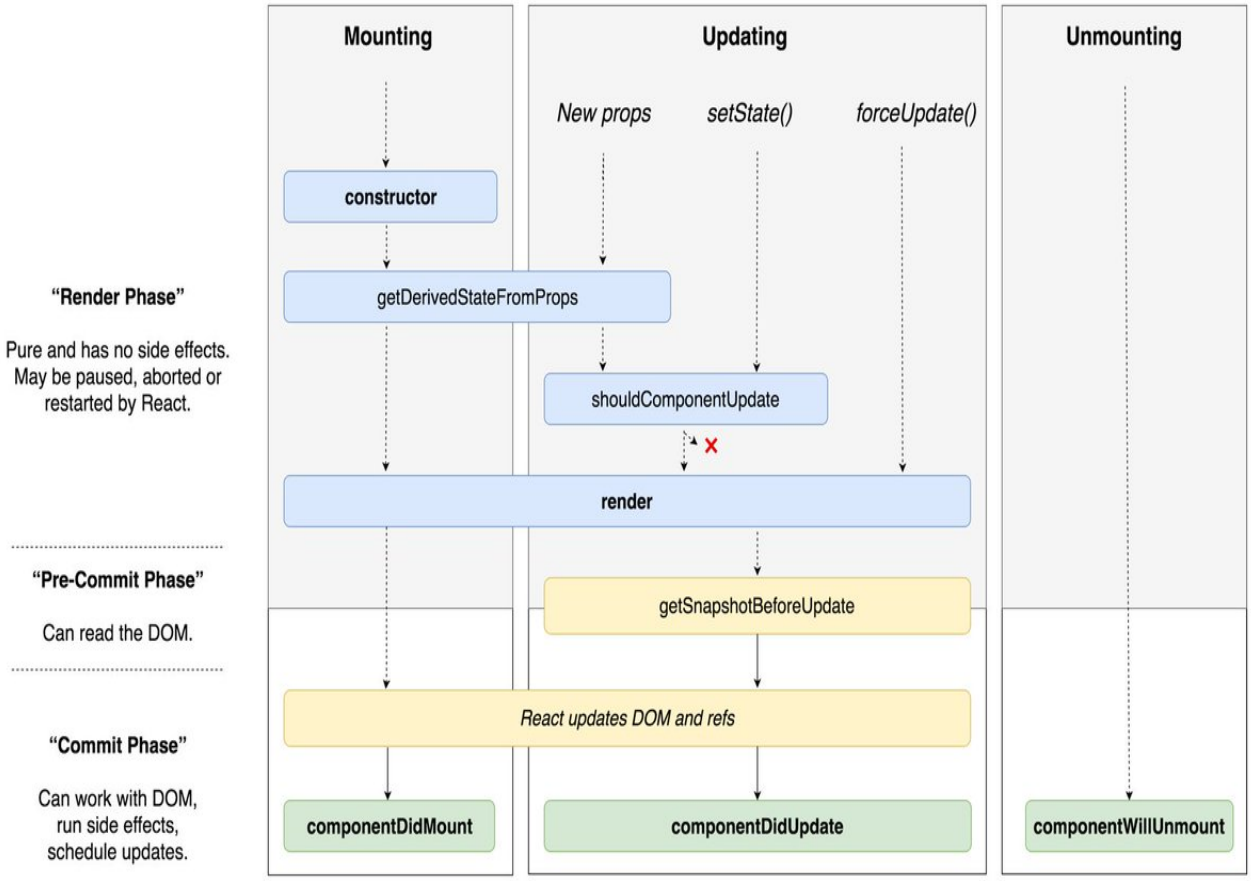
Execution Flow When Typing (Query Change)

```
User types character
    ↓
setQuery(...) updates state
    ↓
React schedules render
    ↓
App() runs again
    ↓
useMemo recomputes filteredUsers (if deps changed)
    ↓
JSX returned
    ↓
DOM committed
    ↓
useEffect([query]) runs
```

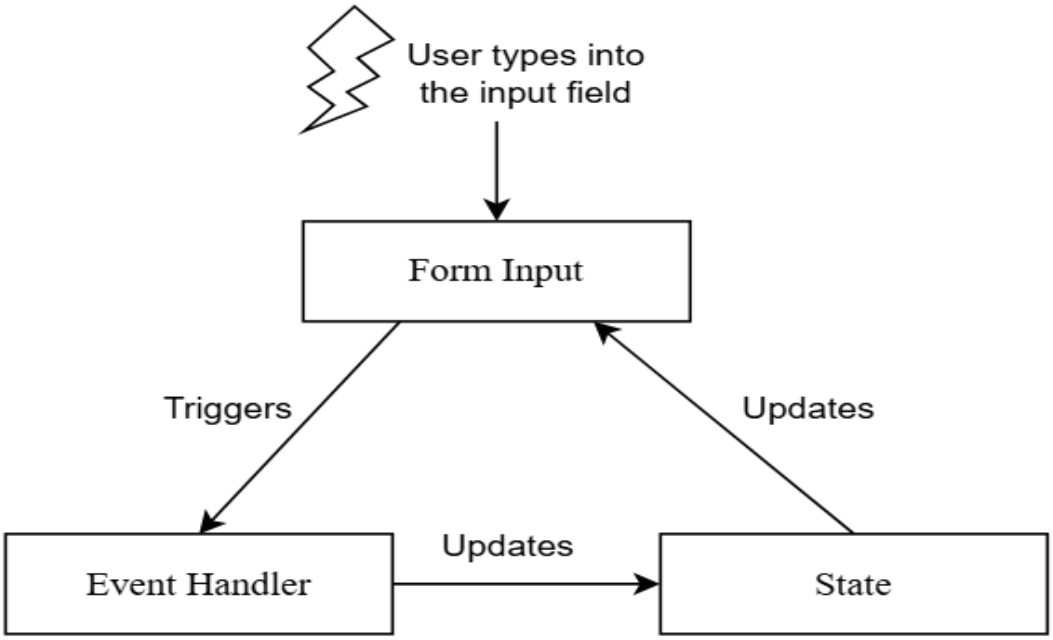
Why filteredUsers Runs Before useEffect

filteredUsers is computed during render because it is derived UI data. useEffect always runs after render and commit. Therefore, filteredUsers can never be triggered by useEffect directly.

Visual Lifecycle Reference



Render vs Effect Separation (Reminder)



These diagrams reinforce the separation: render computes UI, commit paints it, effects observe changes after the paint and may schedule the next render.

Day 6 – State■Driven Drawer (Group → Permissions)

This section explains how the Group Permissions Drawer works using pure state and re-rendering. Visibility and content are entirely driven by React state.

1) What happens when you click a group

```
onClick={() => openGroupDrawer(group.name)}

function openGroupDrawer(groupName) {
  const mapping = user.groupToPermissions.find(
    (m) => m.group === groupName
  );

  setDrawerGroup(mapping || { group: groupName, permissions: [] });
  setDrawerOpen(true);
}
```

Two state updates happen: drawerGroup is populated and drawerOpen becomes true. No UI code runs here — just state.

2) Why the Drawer appears

```
<Drawer anchor="right" open={drawerOpen} onClose={() => setDrawerOpen(false)}>
```

When drawerOpen is true, React re-renders and the Drawer becomes visible.

3) Why the correct group name shows

```
<Typography variant="h6">
  {drawerGroup.group}
</Typography>
```

State drives JSX. When drawerGroup changes, the UI updates automatically.

4) Conditional rendering of permissions

```
{drawerGroup.permissions?.length ? (
  <Box>
    {drawerGroup.permissions.map((code) => (
      <Chip key={code} label={code} />
    ))}
  </Box>
) : (
  <Typography>No permissions found</Typography>
)}
```

This defensive UI ensures the Drawer never crashes and always shows something sensible.

5) Why optional chaining is used

Optional chaining prevents runtime errors when state is temporarily undefined during render.

6) How the Drawer closes

```
onClose={() => setDrawerOpen(false)}
```

Closing the Drawer is just another state update followed by a re-render.

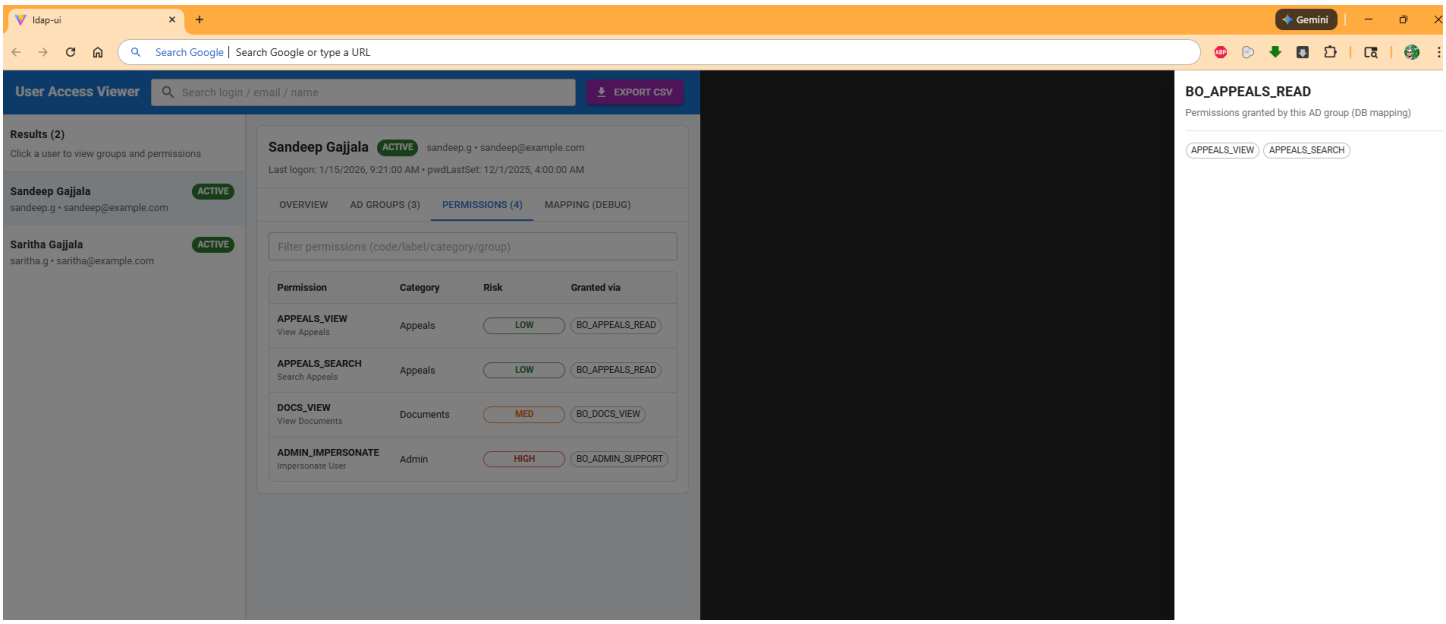
7) Big picture flow

1 User clicks group row

2. openGroupDrawer updates state
3. React re-renders
4. Drawer opens and renders content
5. User closes Drawer → state resets → Drawer hides

This reinforces a core Day■6 pattern: state■driven UI containers such as drawers, modals, and dialogs.

Appendix: Current User Access Viewer UI (Screenshot)



Day 7 – Modularization & NYSOH Theme

Day 7 focused on refactoring the large UserAccessViewer into modular components and introducing a clean API abstraction layer. A global MUI ThemeProvider was added to standardize the UI with NYSOH-style branding.

```
src/
  api/
    userAccessApi.js
  hooks/
    useDebouncedValue.js
  components/
    ResultsPane.jsx
    GroupsTab.jsx
    PermissionsTab.jsx
    MappingTab.jsx
    GroupPermDrawer.jsx
  theme/
    nsyohTheme.js
```

Day 8 – Debounced Search & Backend Readiness

Day 8 implemented production-ready debounced search using a custom hook and AbortController to prevent race conditions when typing rapidly.

```
User types
   ↓
useDebouncedValue (300ms)
   ↓
useEffect runs
   ↓
searchUsers({ signal })
   ↓
AbortController cancels stale request
   ↓
State updates → Re-render
```